



The Brilliant Employee · 04

An analytics job ate my review queue

A review queue read 2 of 5 for days, so the backlog looked like mine. 83 of the 85 rows in the file that gates it were machine output, so the sampling pool was already empty.

sgnk.ai/brilliant ↗

Sagnik Mitra

Why this exists

A language model writes a large share of my production code. Wrong work reads exactly like right work, and no prompt fixes that. So the system below exists to find out when the answers were wrong.

From one week, none of it visible from inside the tool: a sizing step ignored on 796 of 816 tasks, and two fixes written up as done that were never made.

THE SYSTEM, AS OF 7 AUGUST 2026, 14:51Z	
rules written	11 June, 57 days ago
ledger running	27 June, 41 days
tasks logged	3,103, across 3,620 rows
controls on tool calls	6 registered, 4 that can refuse

You can rent the model. You cannot rent the part that tells you it was wrong.

Five parts, and where today sits

Five parts sit around the model. Each post takes one of them.

sizing check	sizes a job before a model is picked
---------------------	--------------------------------------

model picker	recommends which model gets the job
---------------------	-------------------------------------

safety stops	refuse the irreversible, before it runs
---------------------	---

grader	grades finished work pass or fail
---------------	-----------------------------------

task log	at least one append-only line per task
-----------------	--

Listed in the order they act on a task, not the order they were built.

Today is about the grader, marked above.

ASIDE. Only the safety stops can refuse. The other four observe: one advises, one recommends, one grades after the fact, one records.

This page is the map. Nothing here needs another post to make sense.

The dashboard blamed me and I believed it

A Judge, another piece of software, grades my automation system's work. Nobody trusts a grader on faith, so the remedy is written down: the grader counts for nothing until it agrees with me on five human-reviewed examples. I built exactly that.

2 of 5

For days the dashboard sat here. I read it the obvious way: I was the bottleneck, and the system's own status reports agreed, filing it under waiting on the human.

Plain words: HUMAN GOLD. a verdict a human actually made on a real task. Machine-derived rows do not qualify, and the tool's own help text says so.

A requirement of five with a count of two looks like human delay. Check the queue before you blame the human.

Two correct components starved the pipeline

The sampler is the tool that serves items for review. Its rule was sensible: if a task already carries any gold row, it has been judged, stop serving it. That rule is correct exactly as long as gold means what it claims to mean.

THE ANALYTICS PASS

did

ate the pool

behaved

**exactly as
written**

THE SAMPLER

did

**enforced its
rule**

behaved

**exactly as
written**

Meanwhile an analytics pass had been writing mechanical rederivations, machine-recomputed labels, into the same gold file. Every row it wrote made one more task look already reviewed.

Two components, each locally correct,
starved the pipeline between them.

No error was ever raised

- 1 Analytics pass writes machine-derived rows into the gold file, 83 of 85 tasks tagged
- 2 Sampler retires any task carrying any gold row, per its rule
- 3 Sampling pool drains to 0
- 4 Sampler reports 0 unadjudicated. No error, no warning, anywhere
- 5 Dashboard holds at 2 of 5. The human gets blamed

```
# before: any gold row retires a task
done = {r.get("correlation_id")
        for r in gold_rows()}

# after: only a HUMAN verdict does
done = {r.get("correlation_id")
        for r in gold_rows()
        if r.get("source") == "human"}
```

Every step is defensible in isolation, so no single component could report the failure.

A zero from a pipeline is indistinguishable from a broken pipeline until something asks: zero out of what.

Only 2 of the 85 rows were human

83

85

gold rows that were machine output wearing a gold sticker

Only two rows in the entire gold file were verdicts a human had made. The other 83 were mechanical rederivations, analytics the tool's own help text explicitly labels NOT human gold. The sampler could not tell the difference because nothing forced it to.

One notch worse: a second consumer, the report that decides when the grader counts as calibrated, gated correctly on the human count, but the precision it would print once that count reached five was computed over all 85 pooled rows. The trigger was honest. The number behind it would have been mostly machine agreeing with itself.

**The queue was not just blocked. The
counterfeit was about to be counted.**

Even a served item could not be graded

Suppose the pool had never starved and an item reached me. The function written specifically to recover the evidence behind each label was wired into a different subcommand and nothing else. The review screen could not show the work.

WHAT A SERVED SAMPLE ACTUALLY SHOWED	
correlation id	present
timestamp	present
machine verdict, kind, weight	present
files, diffs, output	absent
A verdict with nothing under it. Grade that.	

So the homework was impossible twice over. No items could be served, and a served item could not be graded. Two independent locks, zero alarms between them.

Test the review path end to end as the reviewer, not as the plumber.

The sampling pool went 0 to 83

PRE-FIX

sampling pool

0

human gold count

2

errors raised

0

POST-FIX

sampling pool

83

human gold count

still 2

set equality

both directions

Measured at the fix commit, 29 July 2026. The 83 rows returning to the pool are exactly the 83 mechanical rows, checked as set equality in both directions. And the human count stays at 2, because those two really are the only human verdicts. A fix that moved that number would have been a second bug. Run the same computation today and the pool is larger, because more machine labels have landed since.

ASIDE. A differential means asking the same question before and after the fix and diffing the answers. Cheap, and it ends arguments.

A good fix changes exactly the numbers
it should, and proves it in both directions.

A zero means nothing without a denominator

The shape generalizes past review queues. The only thing separating a healthy zero from a dead pipeline is the denominator, and who counted it.

SYSTEMS THAT PRINT SUCCESS	
a backup job that writes nothing	prints success
a scanner that finds nothing	prints success
a queue that serves nothing	prints success
Zero out of what, and who counted the what.	

- 1 Every zero gets a denominator before it gets believed
- 2 Every quiet system gets asked what it would look like broken
- 3 gold-pool-not-starved is now a standing gate: it had to prove it could fail before it was trusted to pass

Any queue whose empty state means both done and broken needs a liveness counter, not a status check.



A queue that empties on success and on failure needs a counter.

How a correct sampler and a correct writer combined into silence.
Full dissection.

sgnk.ai/brilliant ↗

Sagnik Mitra

I build AI systems and the machinery that keeps them honest: gates, ledgers, judges, and rails. Product design for years, engineering the whole time. This series is what the building actually taught me.

sgnk.ai · in/sagnikmitra · github.com/sagnikmitra